

什么是 D 进程？

在 Linux 系统中，D 进程（或称为 **Uninterruptible Sleep** 状态的进程）是指那些在内核空间等待某些事件（如磁盘 I/O、网络请求等）完成时无法被中断的进程。通常，D 状态表示进程正在等待 I/O 操作完成，且在此期间无法被正常调度或被信号中断。

D 进程状态解释：

- D 状态的进程处于不可中断的睡眠状态，这种状态的进程通常不会响应 **SIGKILL** 等信号，直到它完成当前的 I/O 操作或者系统事件。
- 进程处于 D 状态时，通常意味着它正在等待磁盘、网络或者其他一些硬件的响应（如等待磁盘的读写完成）。
- 如果一个进程长时间处于 D 状态，通常是由于某些 I/O 操作发生了堵塞，或者硬件资源不可用。

可以通过以下几种方式查看 D 状态的进程：

1. **ps** 命令：

使用 **ps** 命令查看所有进程的状态。

```
ps aux | grep ' D '
```

这个命令将列出所有状态为 D 的进程。

2. **top** 命令：

在 **top** 命令中，进程的状态列会显示进程的当前状态。在 **top** 的输出中，D 表示进程处于不可中断的睡眠状态。

假设你已经确定了进程 PID 为 1234，你可以使用 **perf record** 来收集该进程的性能数据：

```
sudo perf record -p 1234
```

该命令会开始记录进程 1234 的性能数据，直到你按下 **Ctrl + C** 来停止收集。

分析 I/O 问题

对于 D 状态的进程，通常是 I/O 操作造成了阻塞。你可以使用 **perf** 工具来定位与磁盘 I/O 操作相关的性能瓶颈。

1. 使用 **perf trace** 查看系统调用：

perf trace 可以显示系统的各个系统调用和它们的执行时间，帮助你分析进程在哪些 I/O 操作上卡住了。

```
sudo perf trace -p 1234
```

这个命令将显示进程 1234 执行的系统调用，包括所有的 I/O 操作。你可以通过这个输出查看进程是否在等待磁盘操作（如 **read** 或 **write**）等。

2. 分析磁盘 I/O 操作：

如果进程在执行大量磁盘 I/O 操作时卡住，可以使用 **iostat** 或 **dstat** 工具来查看磁盘的 I/O 使用情况，判断是否有磁盘瓶颈：

```
iostat -x 1
```

这些工具会帮助你查看磁盘的读写速率、延迟等信息。

检查磁盘或网络挂起状态

对于长时间处于 D 状态的进程，可能是硬件、网络、磁盘等资源出现了问题。你可以检查磁盘的健康状况，或者网络连接是否出现问题。

1. 磁盘健康检查：

使用 **smartctl** 工具检查磁盘的健康状况：

```
sudo smartctl -a /dev/sda
```

该命令将输出磁盘的健康信息，帮助你判断磁盘是否存在硬件故障。

2. 网络故障检查：

如果是由于网络阻塞导致的进程 D 状态，可以使用 **netstat** 或 **ss** 来查看当前的网络连接状态，看看是否有长时间挂起的网络连接。

```
ss -t -a
```

进一步调优或修复

1. 磁盘 I/O 优化:

如果确定问题是由于磁盘 I/O 操作导致的, 可以考虑优化磁盘的读写性能, 使用 SSD 来代替 HDD, 或者增加磁盘缓存等。

2. 调节 I/O 调度策略:

你还可以调整 I/O 调度器, 使用 `deadline` 或 `noop` 调度器来优化磁盘 I/O:
`sudo echo deadline > /sys/block/sda/queue/scheduler`

3. 调节网络配置:

如果是网络问题, 可能需要检查网络的带宽、延迟等, 并优化网络配置。

用户访问 `https://a.com` 突然无响应 (打不开 / 超时 / 502 / 500 等)

我们从客户端 → DNS → 网络 → LB → 应用服务器 → 数据库一层层完整排查。

永远按顺序:

客户端

→ DNS

→ 网络

→ LB

→ 应用

→ 依赖服务 (DB/Redis/第三方)

→ 系统资源

下面给你一套接近真实生产环境的完整 Debug Playbook (偏 Linux 环境)

第一阶段: 确认现象 (外部视角)

确认是否真的无法访问

`curl -v https://a.com`

作用

- 看是否能建立 TCP 连接

- 是否 TLS 握手成功
- 是否返回 HTTP 状态码
- 是否超时

可能结果

现象	可能问题
Could not resolve host	DNS 问题
Connection refused	端口没监听
Connection timed out	网络 / 防火墙
502/504	上游挂了
500	应用内部错误

第二阶段：DNS 排查

检查 DNS 解析是否正常

`dig a.com`
或

`nslookup a.com`
作用

- 查看是否解析出 IP
- TTL 是否异常
- 是否被解析到错误 IP

检查是否全球 DNS 问题

```
dig @8.8.8.8 a.com
```

```
dig @1.1.1.1 a.com
```

作用

排除本地 DNS 污染

检查域名是否过期

```
whois a.com
```

作用

查看是否过期

第三阶段：网络连通性排查

假设 DNS 解析到：

```
a.com -> 1.2.3.4
```

测试 IP 是否可达

```
ping 1.2.3.4
```

⚠ 注意：很多生产环境禁 ping，ping 不通不代表一定挂。

检查端口是否开放

```
nc -zv 1.2.3.4 443
```

或

```
telnet 1.2.3.4 443
```

作用

- 检查 TCP 三次握手是否成功

如果这里卡住：

- 防火墙
- 安全组
- 服务器没监听

路由追踪

```
traceroute 1.2.3.4
```

或

```
mtr 1.2.3.4
```

作用

查看是否网络中断

第四阶段：Load Balancer 排查

假设 1.2.3.4 是 LB

登录 LB 机器

```
ssh lb-server
```

查看 LB 是否在监听端口

```
netstat -tlnp | grep 443
```

或

```
ss -tlnp | grep 443
```

作用

确认 LB 进程是否运行

检查 LB 日志

Nginx:

```
tail -f /var/log/nginx/error.log
```

HAProxy:

```
tail -f /var/log/haproxy.log
```

看什么?

- upstream timed out
- no live upstream
- connection refused

检查后端健康状态

如果是 Nginx upstream:

```
curl http://backend1:8080/health
```

或直接访问后端 IP:

```
curl http://10.0.0.11:8080
```

如果访问后端失败，问题在应用服务器。

第五阶段：应用服务器排查

假设 backend IP 是:

```
10.0.0.11
```

登录应用服务器

```
ssh app-server
```

检查服务是否运行

```
ps aux | grep java
```

或

```
systemctl status myapp
```

检查端口监听

```
ss -tlnp | grep 8080
```

本机 **curl** 测试

```
curl localhost:8080
```

如果本机都失败：

- 应用挂了
- 端口没启动
- 程序卡死

查看应用日志

```
tail -f /var/log/myapp/app.log
```

重点看：

- OOM
- DB connection timeout
- NullPointerException
- thread pool exhausted

查看资源使用情况

```
top
```

或

```
htop
```

检查：

- CPU 100%
- 内存打满
- Load average 很高

查看文件句柄是否耗尽

```
ulimit -n  
lsof | wc -l
```

查看线程数

```
ps -eLf | wc -l
```

第六阶段：数据库排查

如果应用日志报：

```
DB connection timeout
```

测试数据库连通性

```
telnet db-ip 3306  
或
```

```
nc -zv db-ip 3306
```

登录数据库

```
mysql -h db-ip -u user -p
```

查看当前连接数

MySQL:

```
SHOW PROCESSLIST;  
SHOW VARIABLES LIKE 'max_connections';
```

查看慢查询

```
SHOW FULL PROCESSLIST;
```

查看数据库负载

在 DB 服务器：

```
top  
iostat -x 1  
看：
```

- IO 100%
- CPU 打满
- 等锁

高阶排查工具（进阶）

抓包

```
tcpdump -i eth0 port 443  
查看是否收到请求
```

查看连接状态

```
ss -s
```

查看 TIME_WAIT 是否爆炸

```
netstat -an | grep TIME_WAIT | wc -l
```

LoadAvg的常见问题排查

| 现象 | 真相 |

| ----- | ----- |

| load 高 CPU 高 | 算力瓶颈 |

| load 高 CPU 低 wa 高 | IO 问题 |

| load 高 CPU 低 wa 低 | 线程阻塞 / 锁 |

| load 高但流量不大 | 死锁 / bug |

稳定系统必须具备四个机制：

限流

熔断

隔离

降级

限流（防止过载）

核心思想：

不能让流量无限进入

方式：

- 网关限流（Nginx / Kong）
- 应用限流（Sentinel）
- 令牌桶
- 漏桶

目的：

系统处理能力 1000 QPS

就绝不允许 2000 QPS 进入

熔断（防止等待放大）

当下游错误率高时：

直接失败，不再调用。

例如：

- 连续 50% 失败
- 直接 10 秒内不再请求

避免：

线程全部阻塞

隔离（防止相互拖死）

必须做：

- 线程池隔离
- 连接池隔离
- 资源隔离

例如：

支付线程池 50

库存线程池 30

日志线程池 10

否则：

一个慢接口拖死所有接口。

降级（优雅失败）

例如：

- 商品详情页
- 推荐系统挂了

可以：

返回默认推荐
而不是 500。

如何设计一个“不会雪崩”的系统（架构级设计）

我们从底层原则讲，而不是工具。

核心原则一：系统必须有“上限”

所有雪崩的根因只有一个：

没有边界。

必须有的边界：

- 最大并发数
- 最大线程数
- 最大队列长度
- 最大连接数
- 最大缓存大小
- 最大 QPS

如果有任何一个是“无限”：

```
LinkedBlockingQueue()  
newCachedThreadPool()
```

无限重试
无 timeout
迟早雪崩。

核心原则二：必须有“背压”

什么是背压？

当系统忙不过来时：

必须拒绝新请求，而不是继续堆积。

例如：

- 线程池队列满 → 拒绝
- DB 连接池满 → 失败
- Redis 慢 → 降级

而不是：

- 一直等
- 一直排队
- 一直重试

核心原则三：必须“隔离”

雪崩的本质是：

一个模块拖死全部模块。

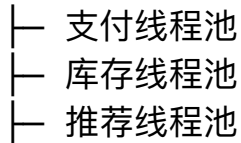
解决方案：

- 线程池隔离
- 连接池隔离

- 缓存隔离
- 服务拆分

例如：

订单服务



库存慢，支付不能受影响。

核心原则四：必须“超时”

没有超时 = 等死。

所有远程调用必须：

`connectTimeout`

`readTimeout`

整体调用超时

否则：

线程会永久卡住。

核心原则五：必须“降级”

例如：

- 推荐挂了 → 返回默认推荐
- 评论挂了 → 不展示评论
- 风控慢 → 走简单规则

核心路径必须保证：

下单 ≠ 被推荐系统拖死

核心原则六：故障可控 + 不扩散 + 自动恢复

健康系统：

QPS ↑

延迟 ↑

部分拒绝 ↑

但系统仍然可用
